2026년도 캡스톤디자인 포트폴리오

실시간 로켓 시뮬레이터 및 텔레메트리 모니터링 시스템

팀명 : SpaceK
지도교수 : 정환익
팀장 : Sergei Kukharenko
팀원 : Komal, Aruzhan, Ruslan

Table of Contents

01. 프로젝트 개요

02. 시스템 구성

03. 시스템 구성 설명

04. 시스템 운영 설계

05. 시스템 활용 방안

06. 기대 효과 및 결론

1. 프로젝트 개요

실시간 로켓 시뮬레이터 및 텔레메트리 모니터링 시스템

프로젝트 개요:

C++로 Falcon 9 비행을
시뮬레이션하고,
물리 제어 패널로 조작

고도·속도·질량·각도
데이터를 웹 대시보드에서
실시간 시각화

기대 효과:

01. 로켓 비행 원리의
직관적인 이해

02. 실제 관제 시스템과
유사한 모니터링 경험

03. 비행 기록 저장·재생
을
통한 데이터 분석

1. 팀원 및 담당 분야

과제명: 실시간 로켓 시뮬레이터 개발

Sergei
(팀장)

C++ 비행 시뮬레이터 제어 패널

C++ · Raspberry Pi · Wemos

Komal
(팀원)

실시간 웹 대시보드 Node.js 브리지

React · TypeScript · Node.js

Aruzhan
(팀원)

Replay 페이지

React · TypeScript

Ruslan
(팀원)

텔레메트리 DB 구축

MySQL · Python

제어 패널로 명령 → C++ 시뮬레이터 → 웹·DB 실시간 확인

1. 팀원별 진행 내용

Sergei Kukharenko (팀장)

C++ 로켓 비행 시뮬레이터 및 제어 패널 개발

C++ · CMake
OpenGL · ImGui
Pi 4· Wemos

관련 업무 소개

Falcon 9 2단 로켓 2D 비행 시뮬레이터 (대기·공기역학, 자동 단계 분리)

업무 추진 방법

C++ 물리 계산, 멀티스레드 구조

업무 추진 내용

- 2단 물리 모델·단계분리
- Wemos 펌웨어 (Boost.Asio)
- OpenGL+ImGui 제어 화면
- 텔레메트리 UI·DB 전송

1. 팀원별 진행 내용

Komal
(팀원)

실시간 텔레메트리 웹 대시보드 및 Node.js 브리지 개발

React · TypeScript
Tailwind · Recharts
WebSocket · Node.js

관련 업무 소개

라즈베리파이의 텔레메트리 데이터를 실시간 그래프와 수치로 표시하는 웹 대시보드 개발

업무 추진 방법

React + TypeScript로 UI 구성, WebSocket으로 데이터 수신, Recharts로 그래프 시각화

업무 추진 내용

- Node.js 브리지 (TCP → WebSocket 변환)
- WebSocket 연결 및 TELEMETRY 요청 수신
- 5개 항목 실시간 그래프 및 연료량(%) 계산
- 비행 기록 JSON 파일 내보내기

1. 팀원별 진행 내용

Omarkhanova Aruzhan (팀원)

비행 기록 재생(Replay) 및 사후 분석 페이지 개발

React · TypeScript
Tailwind · Canvas 2D API

관련 업무 소개

JSON으로 저장된 비행 데이터를 그래프와 계기판으로 재현하여 사후 비행 분석 지원

업무 추진 방법

외부 그래프 라이브러리 없이 Canvas 2D API로 직접 시각화, React 상태와 타이머로 타임라인 제어

업무 추진 내용

- JSON 파일 업로드 및 유효성 검증
- 1x · 2x · 5x 배속 재생 및 타임라인 이동
- 4분할 카드형 실시간 계기판
- Canvas 기반 비행 곡선 차트 및 타임라인 가이드

1. 팀원별 진행 내용

Li Ruslan (팀원)

로켓 텔레메트리 데이터베이스 설계 및 구축

MySQL · Python
MySQL Workbench
· TCP/IP

관련 업무 소개

라즈베리파이로부터 수신한 고도·거리·속도·질량·각도 데이터를 MySQL에 저장하고 관리

업무 추진 방법

Python으로 JSON 텔레메트리를 분류하고, launch_id와 sample_no 기준으로 MySQL에 저장

업무 추진 내용

- TCP 통신으로 실시간 텔레메트리 수신
- Python–MySQL 데이터베이스 연동
- 발사·항목별 테이블 및 분석용 통합 테이블 설계
- 발사별 데이터 저장 및 상태 관리

1.개발 목적 및 필요성

교육용 도구

고도·속도·질량 변화를
시각적으로 이해

실시간 관제 경험

실제 발사 관제 센터처럼
실시간 데이터 모니터링

HW + SW 결합

제어 패널과 시뮬레이터·
서버·웹·DB 통합

비행 데이터 분석

비행 기록 저장 및
재생으로 사후 분석

1. 유사 제품 조사 및 기술 동향

SpaceX 미션 컨트롤
실시간 텔레메트리를
웹으로 공개

Kerbal Space Program
물리 기반 로켓 시뮬레이션
게임, 교육용으로 활용

기술 동향

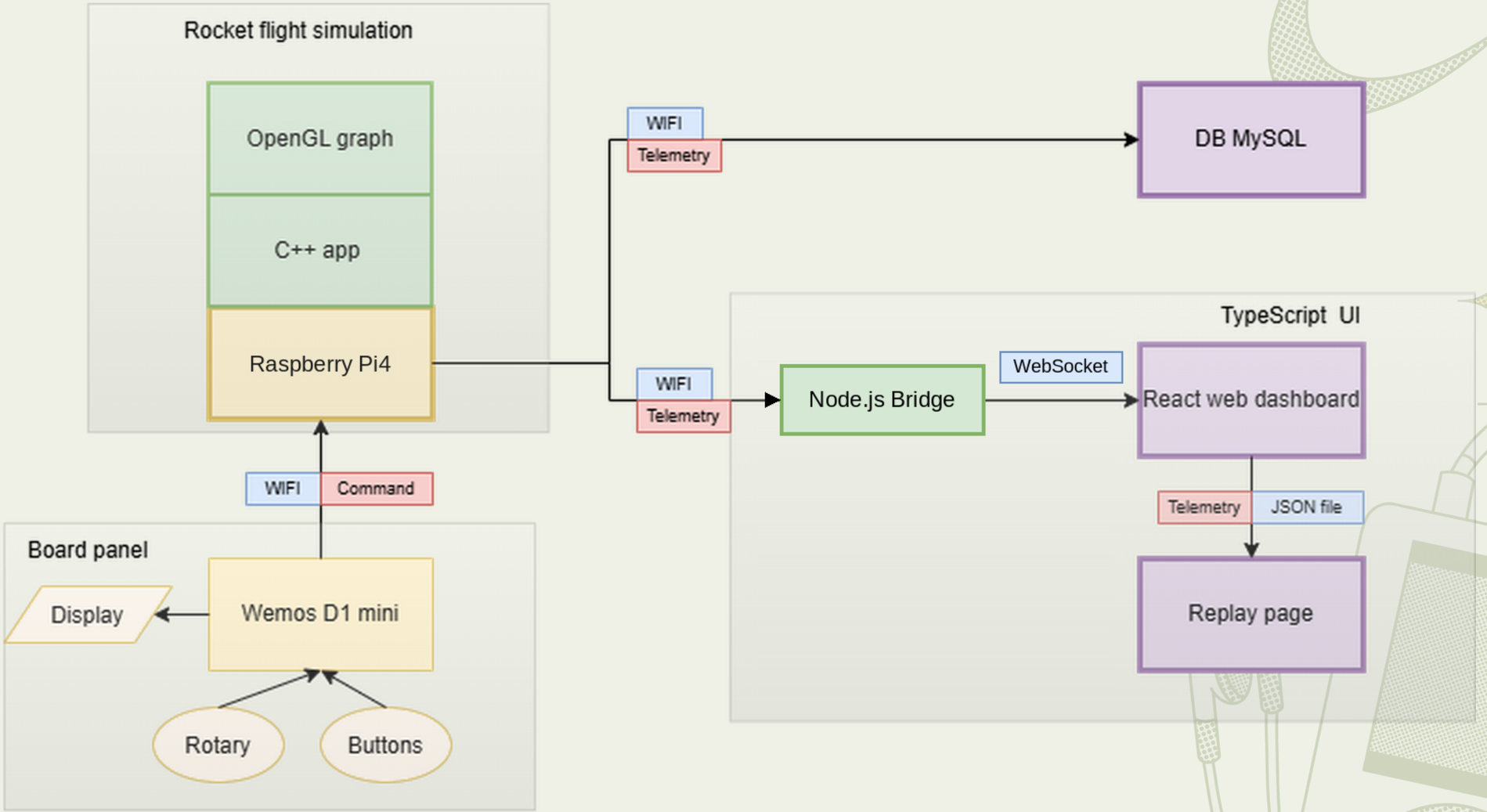
IoT 실시간 모니터링 ·WebSocket 저지연 전송
·웹 데이터 시각화

차별점: 물리 제어 패널 + C++ 시뮬레이션 + 웹 대시보드 + DB + 비행 재생

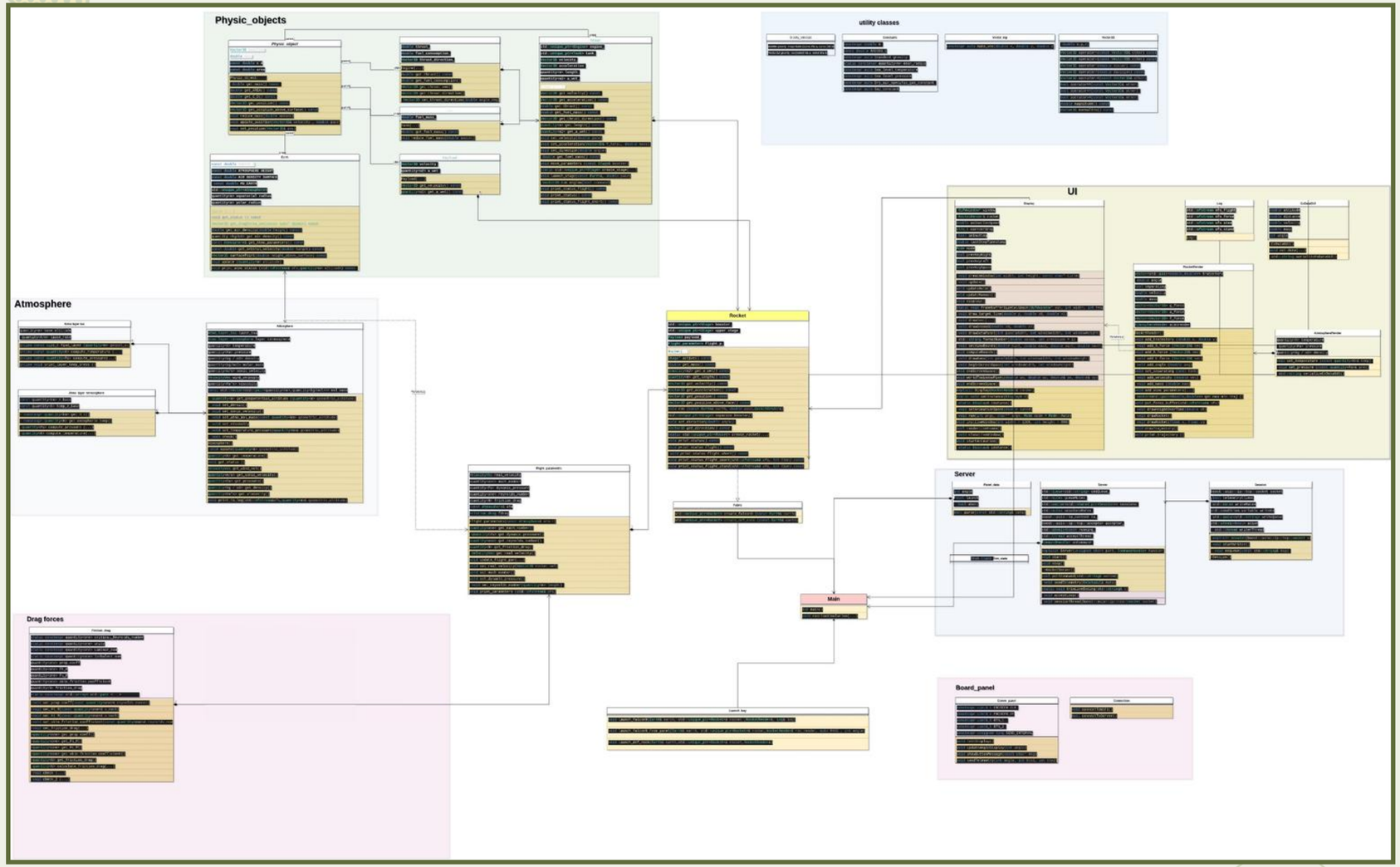
1. 시스템 구성 (레이어)

① 제어 패널	Wemos D1 mini · 버튼 · 로터리
② 시뮬레이션	Raspberry Pi 4 · C++ · OpenGL
③ 통신	WiFi · TCP · Node.js 브리지 · WebSocket
④ 데이터 저장	MySQL
⑤ 사용자 인터페이스	React 대시보드 · Replay 페이지

1. 시스템 구성도


Rocket flight simulation
OpenGL graph
C++ app
Raspberry Pi4
WIFI
Telemetry
DB MySQL
WIFI
Telemetry
Node.js Bridge
WebSocket
TypeScript UI
React web dashboard
Telemetry
JSON file
Replay page
WIFI
Command
Board panel
Display
Wemos D1 mini
Rotary
Buttons

1. 시뮬레이션 UML 클래스 다이어그램



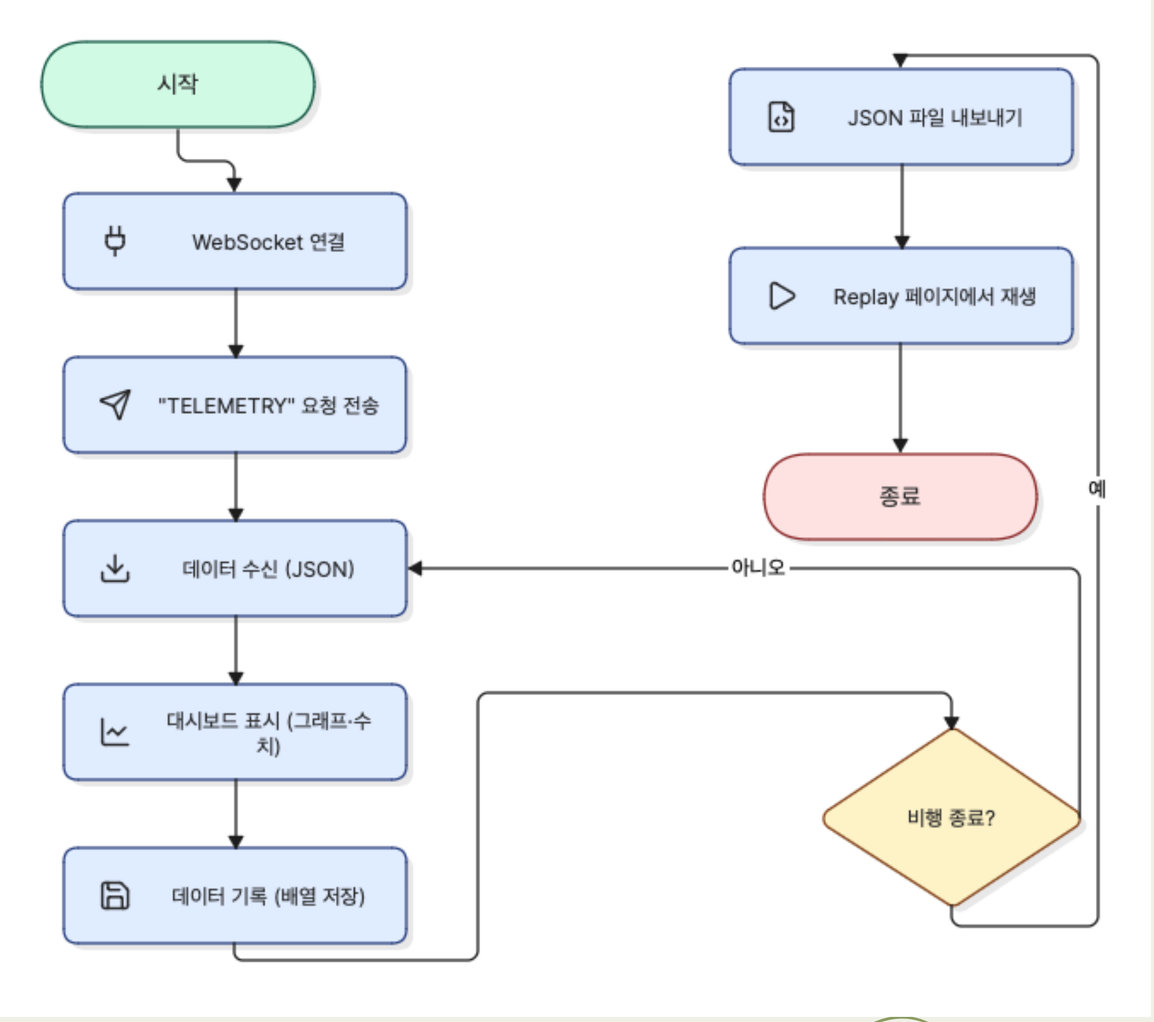
- Physic_objects: 물리 객체
- Atmosphere: 대기 모델
- Drag forces: 항력 계산
- UI: 화면 렌더링
- Server: 서버 통신
- Board_panel: 제어 패널

"총 30+ 클래스, C++ 객체지향 설계"

1. 레이어 설명

① 제어 패널	버튼·로터리 입력 → Wemos가 WiFi로 명령 전송
② 시뮬레이션	C++로 Falcon 9 비행 물리 계산 및 OpenGL 시각화
③ 통신	TCP 전송 → Node.js 브리지에서 WebSocket 변환
④ 데이터 저장	텔레메트리를 MySQL에 저장
⑤ 사용자 인터페이스	React 실시간 모니터링 · Replay 비행 재생

1. 시스템 작동 플로우 차트



4. 데이터 구조

필드	의미	단위
altitude	고도	m
distance	이동 거리	m
velocity	속도	m/s
mass	질량	kg
angle	비행 각도	°


```
{ "altitude": 62997.9, "distance": 8.56e-12, "velocity": 1360.17, "mass": 235393,
"angle": 90 }
```


- 라즈베리파이 서버가 평균 약 10ms 간격으로 JSON 데이터를 전송
- 대시보드는 수신한 데이터를 실시간으로 그래프에 반영

4. 데이터베이스 구성

launch_id	launch_name	sample_no	received_at	altitude	distance	velocity	mass	angle
13	Flight_2026-10-06_11-27-39	1	2026-10-06 11:27:39.968	1122.38	153.20	107.50	489202.00	90.00
13	Flight_2026-10-06_11-27-39	2	2026-10-06 11:27:40.020	1124.52	153.38	107.61	489153.00	90.00
13	Flight_2026-10-06_11-27-39	3	2026-10-06 11:27:40.023	1126.67	153.55	107.73	489104.00	90.00
13	Flight_2026-10-06_11-27-39	4	2026-10-06 11:27:40.025	1128.82	153.73	107.84	489056.00	90.00
13	Flight_2026-10-06_11-27-39	5	2026-10-06 11:27:40.028	1130.97	153.90	107.95	489007.00	90.00
13	Flight_2026-10-06_11-27-39	6	2026-10-06 11:27:40.030	1133.13	154.07	108.07	488958.00	90.00
13	Flight_2026-10-06_11-27-39	7	2026-10-06 11:27:40.031	1135.28	154.25	108.18	488909.00	90.00
13	Flight_2026-10-06_11-27-39	8	2026-10-06 11:27:40.033	1137.44	154.42	108.29	488860.00	90.00
13	Flight_2026-10-06_11-27-39	9	2026-10-06 11:27:40.074	1139.60	154.60	108.41	488812.00	90.00
13	Flight_2026-10-06_11-27-39	10	2026-10-06 11:27:40.078	1141.77	154.77	108.52	488763.00	90.00
13	Flight_2026-10-06_11-27-39	11	2026-10-06 11:27:40.080	1143.93	154.95	108.64	488714.00	90.00
13	Flight_2026-10-06_11-27-39	12	2026-10-06 11:27:40.082	1146.10	155.12	108.75	488665.00	90.00

- launch_id·launch_name으로 발사별 데이터 구분, sample_no로 수신 순서 저장
- received_at에 수신 시각을 밀리초(ms) 단위까지 기록
- 고도·거리·속도·질량·각도 데이터를 저장하여 비행 분석에 활용

1.시뮬레이션 엔진 및 통신 서버

```
1  Server server(5555, [](const std::string&) {});
2  server.start();
3
4  while (simRunning) {
5      std::string cmd;
6      while (server.pollCommand(cmd)) {
7          panel_data.parse(cmd);
8          if (panel_data.launch == 0 && state == Sim_state::Base)
9              state = Sim_state::Running;
10         if (panel_data.abort == 0)
11             abortRequested = true;
12     }
13
14     if (state == Sim_state::Running) {
15         accumulator += frameTime;
16         while (accumulator >= dt) {
17             launch_bay.launch_falcon9_from_panel(
18                 earth, rocket, render, dt*2, panel_data.angle, exdata);
19             server.sendTelemetry(exdata);
20             accumulator -= dt;
21         }
22     }
23 }
```

[1–2행] 포트 5555에서 C++ TCP 서버 실행

[6–11행] 패널 명령(launch·abort·angle) 처리

[14–18행] 고정 시간 간격(dt)으로 비행 물리 계산

[19행] 매 단계 텔레메트리 전송

Node.js 브리지에서 WebSocket으로 변환 → 웹 대시보드

4. 대시보드 코드

```
1  function connect() {
2    socket = new WebSocket(BRIDGE_URL)
3
4    socket.onopen = () => {
5      socket.send("TELEMETRY")
6      setConnected(true)
7    }
8
9    socket.onmessage = (event) => {
10     const raw = JSON.parse(event.data)
11     const fuel = raw.mass - DRY_MASS
12     setData({
13       H: raw.altitude,
14       V: raw.velocity,
15       F: fuel,
16       fuelPct: (fuel / (START_MASS - DRY_MASS)) * 100,
17       angle: 90 - raw.angle,
18       distance: raw.distance,
19     })
20   }
21 }
```

[4–7행] WebSocket 연결 후 "TELEMETRY" 요청으로 데이터 수신 시작

[11·16행] 질량(mass)으로 연료량 및 연료 잔량(%) 계산

[12–19행] 변환한 데이터를 state에 저장 → 그래프 실시간 업데이트

BRIDGE_URL: Node.js 브리지 주소 (TCP → WebSocket)

4. 트러블슈팅

1.TCP → WebSocket 변환

문제:

라즈베리파이 서버가 TCP 소켓으로 데이터를 전송하여 웹 브라우저에서 직접 연결 불가

원인:

브라우저는 보안상 TCP 소켓에 직접 접속 불가 (HTTP·WebSocket만 지원)

해결:

Node.js 브리지 서버로 TCP 데이터를 WebSocket으로 변환하여 대시보드에 전달

[라즈베리파이] —TCP→ [Node.js 브리지] —WebSocket→ [React 대시보드]

4. 트러블슈팅

1.멀티스레드 통신 처리

문제:

서버 통신과 시뮬레이션을 동시에 실행해야 함 (C++ 멀티스레드 첫 경험)

원인:

서버 스레드와 메인 루프가 같은 데이터에 동시 접근 시 충돌 위험

해결:

명령을 메시지 큐에 저장 → 메인 루프에서 pollCommand()로 하나씩 처리

[서버 스레드] —명령→ [메시지 큐] —pollCommand()→ [메인 루프]

4. 트러블슈팅

1.시뮬레이션·통신·렌더링 주기 동기화

문제:

시뮬레이션·텔레메트리·렌더링의 실행 주기가 달라 성능 불균형 발생

원인:

환경에 따라 프레임 속도가 달라져 시뮬레이션 속도가 불안정

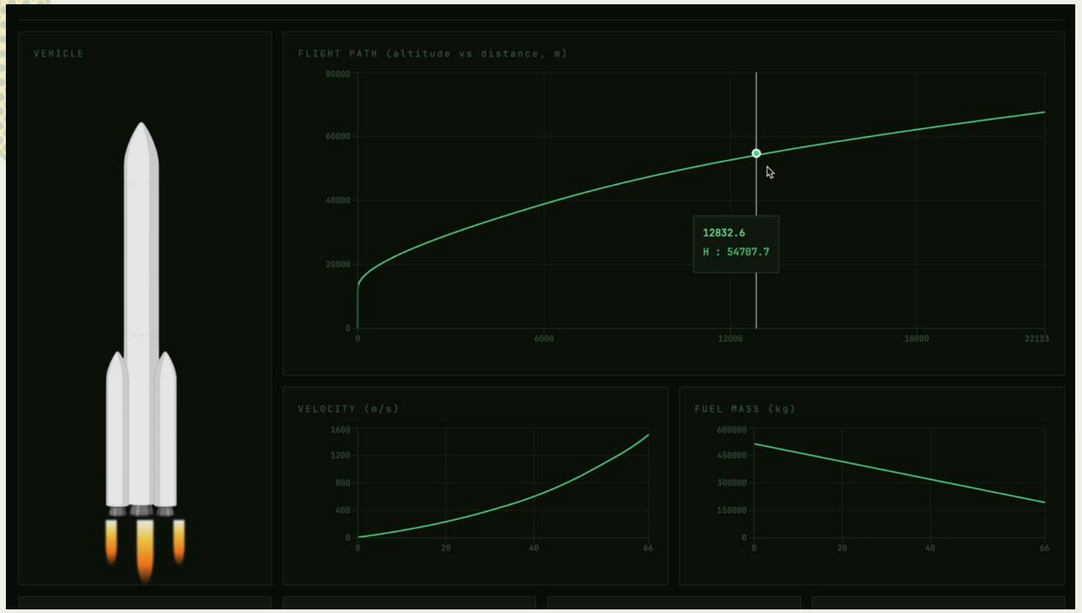
해결:

고정 시간 간격(dt = 0.01초)으로 계산, 프레임 상한 0.025초, 루프마다 5ms 대기

dt = 0.01초 · 프레임 상한 0.025초 · 5ms 대기

4. 웹 UI

실시간 대시보드



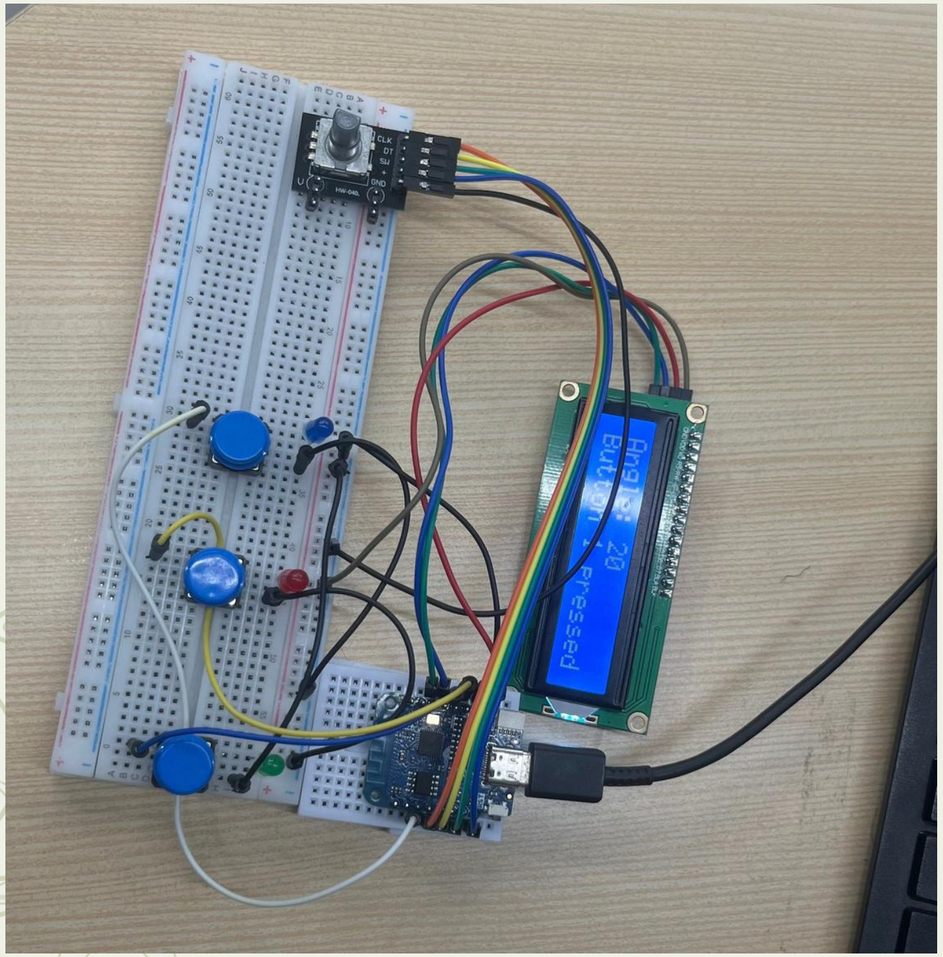


Replay 페이지

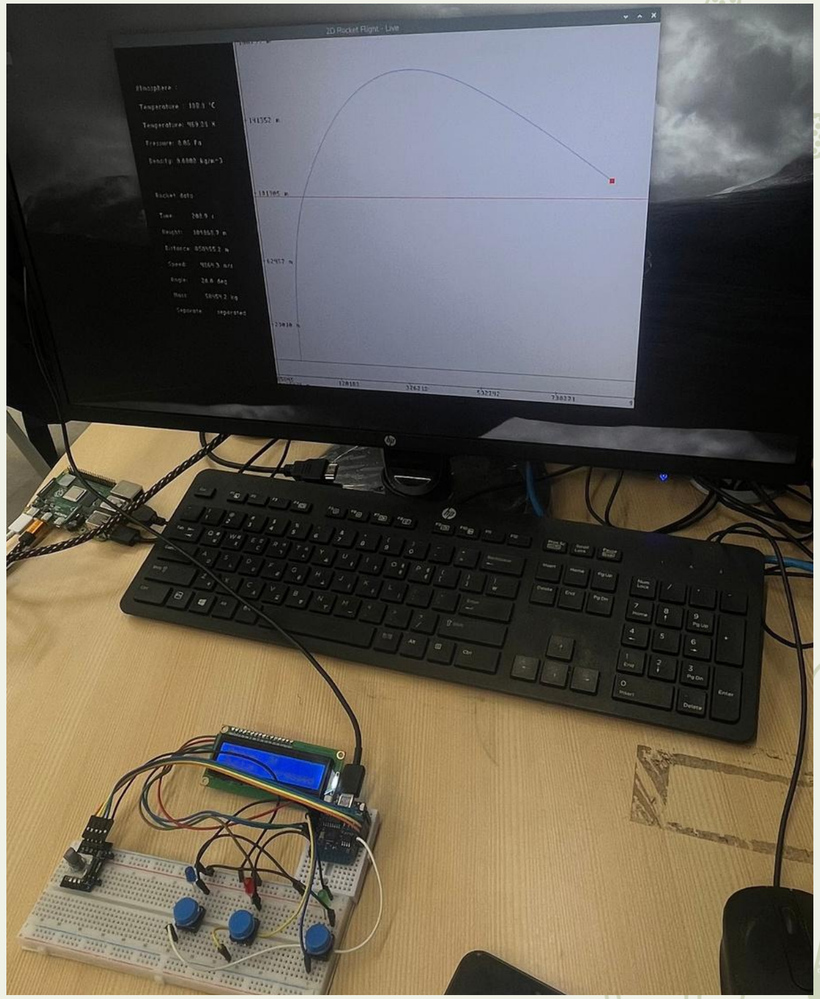




4. 하드웨어 구성



제어 패널 (Wemos D1 mini)



라즈베리파이 시뮬레이터 실행 화면

5. 시스템 활용 방안

교육용 시뮬레이터

고도·속도·연료
변화로 물리
원리 설명

동아리·과학관 체험

발사부터 종료까지
관제 센터처럼
모니터링 체험

IoT 학습 교재

제어 패널 → WiFi
→ 서버 → 웹·DB
구조 학습

6. 기대 효과 및 결론

교육 효과
로켓 비행 원리를
데이터와 그래프
로 직관적 이해

기술적 성과
제어 패널·시뮬
레이터·서버·웹
·DB 통합 구현

확장 가능성
로켓 모델 추가
실제 센서 연동
데이터 분석

회고 하드웨어·시뮬레이션·웹·DB를 연결하며
팀원 간 데이터 형식과 통신 규칙 통일의 중요성을 배움



감사합니다.

팀명 : SpaceK
지도교수 : 정환익
팀장 : Sergei Kukharenko
팀원 : Komal,Aruzhan,Ruslan